

Verifying the Inspect Right: A Standalone Operational Test Procedure for the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone deployment-verification procedure, the operational test that verifies whether a given system implements the **inspect right** as the source paper specifies it. The architectural content of the right is defended in the source paper (§3.1, §3.3, §11.3) and formalized as a standalone commitment in a separate derivation note (A2.01); this note's contribution is the procedure by which a candidate deployment can be verified against that commitment in operational form.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "human-governed" commitment names three architectural rights — to inspect, to modify, and to override substrate content and orchestration rules at any time. A separate note (A2.01) formalizes the inspect right as a standalone architectural commitment with independent operational content. This note formalizes the **operational test** that verifies whether a given deployment satisfies that commitment. The test is not a checklist of qualitative properties; it is a procedural specification — a sequence of steps a deployment verifier can run against a candidate substrate, producing a pass/fail signal with concrete meaning. The note states which architectural commitment the test verifies, specifies the test as numbered operational steps, defines the pass/fail criteria, enumerates the anti-patterns the test specifically detects (LLM-gatekeeping is canonical), describes how the test integrates with the deployment-verification workflow, and bounds the test's scope (what it does NOT verify). The test is the first of a Phase A5 sequence: each subsequent note formalizes the operational test for a different architectural commitment. Comprehensive deployment verification runs all such tests; this note's contribution is one of them, scoped narrowly and named as such.

1. Why the inspect-right test needs to be formalized as standalone

The CKS pattern's "human-governed" commitment is the architectural anchor for the substrate's role in coordination work. Among the three rights that compose the commitment — inspect, modify, override — the inspect right is the most fundamental operationally: a modify or override action taken without the prior ability to read what is being changed is not governance in any robust sense. A separate derivation note (A2.01) formalizes the inspect right's content as a standalone architectural commitment; what that note specifies is *what* the right is. What this note specifies is *how to verify* that a deployment satisfies it.

The two specifications are not the same artifact. A specification of architectural content is a definition; a specification of an operational test is a procedure. A definition lets implementers and reviewers agree on what the right means. A procedure lets a deployment verifier produce a pass/fail signal that is reproducible across deployments, comparable across vendors, and useful as evidence in audit, regulatory review, or composition-partner verification.

Without the procedure named explicitly, deployment verification slides toward qualitative assessment: a reviewer reads a system's documentation, decides on impression whether the inspect right is "implemented," and produces a narrative judgment that downstream parties cannot reproduce. This is not the failure mode a defensible architecture wants. A standalone test, named and scoped narrowly, gives downstream verification a concrete artifact to run, not a feeling to render.

This note opens Phase A5 — the operational-tests-as-standalone phase of the derivation series — with the test for the most fundamental governance right, on the same logic the source paper deploys when it states inspect first among the three rights it commits to.

2. The architectural commitment under test

The test verifies one specific architectural commitment: the inspect right as decomposed from the human-governed commitment of the source paper (§3.3). The commitment has the following content, drawn from the source paper and from the standalone formalization in A2.01.

Read access to substrate content. A human with appropriate access can read any substrate content the deployment authorizes them to see — entities, relationships, decisions, rationale, conflicts, provenance — as the substrate actually carries it.

Read access to orchestration rules. A human with appropriate access can read any orchestration rule that governs cell-level behavior, including the rule's authorship, version history, and current state.

Direct access without LLM intermediation as a precondition. The human may use LLM-assisted tooling on top of the substrate, but the human must also be able to read the underlying substrate content directly, in inspectable form, when they choose. The LLM is permissible as an adjacent tool; it cannot be the gate.

At-the-time-of-choosing access without scheduling or approval gating. The right is exercisable when the human decides to exercise it, not when a workflow permits it. Scheduled review windows, approval-gated sessions, and time-locked inspection portals all fail the temporal component, even when they grant ample read access otherwise.

The commitment is **architectural** rather than procedural — it must be a property of the system's design, not a vendor policy that could change without architectural consequence. The commitment is **temporal** rather than scheduled — it must be available at the human's chosen time, not at predetermined checkpoints. These two qualifiers are themselves the subject of separate derivation notes; the test inherits them as preconditions for what counts as the right being satisfied.

The test does not redefine the commitment; it specifies the procedure that verifies the commitment so defined.

3. The test procedure

The procedure has four steps, performed in order. Each step has a definite outcome that determines whether the step passes; the test as a whole passes only if every step passes.

Step 1 — Authorized read of substrate content sample. The verifier identifies a representative sample of substrate content (entities, decisions, rationale, conflicts, provenance) that an authorized human is entitled to read under the deployment's authorization scope. The verifier or an authorized human attempts to read each item in the sample directly. *Step passes* if every item in the sample is returned, in inspectable form, as the substrate carries it.

Step 2 — Authorized read of orchestration rules. The verifier identifies a representative sample of orchestration rules in force in the deployment. The verifier or an authorized human attempts to read each rule directly, including its authorship, version history, and current state. *Step passes* if every rule in the sample is returned, in inspectable form, as the substrate carries it.

Step 3 — Direct-access verification. For each item read in Steps 1 and 2, the verifier confirms that the read path did not require an LLM call as a precondition — that is, that the content returned was the substrate's actual representation, not an LLM-generated summary, paraphrase, or transformation of it. The verifier may compare the directly-read content against the substrate's underlying representation (database row, file content, structured record) where the substrate's storage layer is independently inspectable. *Step passes* if every item read in Steps 1 and 2 was returned without LLM intermediation in the read path.

Step 4 — Temporal-access verification. The verifier performs Steps 1 and 2 at a time of the verifier's choosing, with no prior scheduling, no workflow approval, and no time-window restriction. *Step passes* if Steps 1 and 2 succeed under these conditions. If the verifier is unable to perform Steps 1 or 2 at the chosen time because of a scheduling constraint, an approval requirement, or a time-window restriction, Step 4 fails regardless of whether Steps 1 and 2 would succeed under permitted conditions.

The procedure produces a single pass/fail signal: pass only if all four steps pass.

4. What the test outputs

The test outputs a binary pass/fail signal whose meaning is defined as follows.

Pass. The deployment satisfies the inspect right as specified in §2. Authorized humans can read substrate content and orchestration rules directly, in inspectable form, without LLM intermediation as a precondition, at the time of their choosing. Pass does not assert that the substrate's content is correct, that provenance is complete, that the modify or override rights are satisfied, or that any other architectural commitment holds. Pass asserts inspection accessibility and nothing more.

Fail. The deployment does not satisfy the inspect right as specified in §2. At least one of the following is true: authorized humans cannot read the relevant content; the content returned is mediated rather than direct; or the read action is unavailable at the human's chosen time. The specific step that fails identifies which component of the right is violated, and the failure record names that component for downstream remediation.

A fail signal does not assert that the deployment is unusable, unsafe, or non-functional in some other respect. It asserts that the deployment is not CKS-coherent on the inspection axis. Whether the failure is remediable, and how, is a separate question the test does not answer.

5. Anti-patterns the test specifically detects

The test detects the inspect-right-violating anti-patterns formalized in Phase A3 of the derivation series. The detection is concrete: each anti-pattern produces a specific step failure.

LLM-gatekeeping (A3.04). When inspection results pass through LLM transformation, summarization, or filtering as a precondition, Step 3 fails. This is the canonical anti-pattern the test detects. A system in which the only path to substrate content is a chat interface or natural-language query layer that returns LLM-generated responses fails Step 3 even if the responses are accurate, because the test verifies *direct* access, not *accurate* access.

Scheduled-review-window inspection (A3.02). When inspection is permitted only during scheduled windows, audits at predetermined intervals, or approval-gated sessions, Step 4 fails. The temporal qualifier makes the failure architectural rather than procedural: a system whose inspection availability is bounded in time fails the test even if the windows are frequent.

Workflow-approval-gated inspection (A3.03). When inspection requires a workflow approval step before substrate content is returned, Step 4 fails. The failure is identical in shape to the scheduled-window case: the gate prevents at-the-time-of-choosing access regardless of how quickly the gate normally clears.

Vendor-managed UI mediation. When the host environment, vendor portal, or runtime middleware transforms substrate content in inspection results — filtering fields, applying access projections that do not correspond to underlying authorization, or altering representation in ways that cannot be reversed by the human — Step 3 fails. The substrate's actual representation must be reachable; a vendor-mediated view that cannot be bypassed is itself the gate.

Per-user-permission-gating beyond authorization scope. When inspection requires roles or permissions beyond the general human-governance authorization scope — for example, when reading substrate content requires acquisition of a separate inspection-only credential issued case-by-case — Step 1 or Step 2 fails for ordinary authorized humans, not because the deployment refuses inspection but because it routes inspection through a separate authority that does not match the architectural commitment.

The five anti-patterns are not exhaustive of all ways a deployment can fail the test, but they are the recurring shapes the test is designed to surface. A failure outside these shapes is still a failure; the architecture grants no exemption for novel failure modes.

6. Integration with deployment verification

The test is designed to be run at four points in a deployment's lifecycle.

Initial deployment validation. Run the test before the deployment is activated for production use. A failed initial test indicates the deployment cannot honor inspect-right claims as designed; remediation is required before activation rather than after.

Composition-partner verification. When the deployment composes with another substrate (per the composition-requirements commitment of A1.13), run the test against each composition partner's substrate, on the same procedure. The composition's overall inspect-right behavior is bounded by the weakest component on this axis; a passing local test does not entail a passing composite test if a partner fails.

Ongoing audit. Run the test periodically per organizational policy — quarterly, annually, or in response to triggering events. Periodic re-running surfaces drift: a deployment that passed at initial validation may fail later if vendor changes, middleware additions, or workflow modifications introduce mediation, scheduling, or approval gating that did not exist before.

Vendor-migration verification. After any vendor change affecting the substrate's storage, access path, or runtime middleware, run the test as a post-migration check. The tool-agnosticism commitment (A1.05) makes the substrate portable across vendors, but the inspect right's satisfaction is not automatically preserved across migration; explicit re-verification is what makes the architectural commitment durable across vendor changes rather than only at the original vendor.

The four points are not exhaustive of when the test can be run, but they are the cases where running it is architecturally indicated. Other cases — incident response, regulatory inquiry, composition-partner negotiation — call the test as appropriate without changing its content.

7. Limits of the test

The test verifies one architectural commitment specifically. Stating what it does not verify is what keeps its scope narrow and its pass signal interpretable.

It does not verify the modify right. A deployment may pass the inspect-right test and fail the modify-right test, or vice versa. The two rights are separable, and their tests are separate (the modify-right test is A5.02).

It does not verify the override right. Override is the third of the three rights; like modify, it has its own test (A5.03).

It does not verify orchestration-rule authoring. Whether orchestration rules are authored by humans is a separate question with its own test (A5.04). The inspect-right test verifies that humans can read rules; it does not verify who wrote them.

It does not verify substrate-content correctness. The test verifies that what is returned matches what the substrate carries; it does not verify that what the substrate carries is true, complete, or up to date. Content correctness is a different concern at a different layer.

It does not verify provenance completeness. Whether each substrate item carries the six-field provenance metadata the source paper commits to is the subject of a separate test (A5.08). The inspect-right test verifies that provenance is *readable*; the provenance-completeness test verifies that provenance is *complete*.

It does not verify substrate authority. Whether the substrate is the source of truth — and whether all five categories of authoritative state live in the substrate rather than in agent memory or vendor-side storage — is the subject of a separate test (A5.10).

It does not verify deterministic state behavior. The substrate's deterministic state guarantees have their own tests (A5.06–A5.07). The inspect right makes determinism *observable*, but observability is a different commitment than the determinism it observes.

A deployment that passes the inspect-right test and fails one or more of the other Phase A5 tests is not CKS-coherent overall, even if it is CKS-coherent on the inspection axis. Comprehensive verification runs the whole Phase A5 sequence; this note's contribution is one component of that sequence, scoped to one commitment.

8. The one-sentence test

A deployment satisfies the inspect right if and only if an authorized human can, at a time of their choosing, read any substrate content and any orchestration rule within their authorization scope, in the form the substrate actually carries it, without an LLM call, scheduling, approval, or vendor-managed transformation in the read path.

9. Conclusion

The inspect-right test is the most fundamental of the operational tests Phase A5 formalizes, on the same logic by which the inspect right is the most fundamental of the three rights the human-governed commitment carries: governance over content one cannot read is not governance in any robust sense. Naming the test as standalone — separable from the inspect right's architectural content, separable from the other rights' tests, separable from the broader human-governed commitment — gives downstream deployment verification a concrete artifact to run, anti-patterns it specifically detects, and a pass/fail signal whose meaning is precise.

The test opens Phase A5. Subsequent notes formalize the modify-right test (A5.02), the override-right test (A5.03), the rule-authoring test (A5.04), the mediator-role test (A5.05), the determinism tests (A5.06, A5.07), the provenance-completeness test (A5.08), the four-accountability-questions test (A5.09), the source-of-truth test (A5.10), the tool-agnosticism-migration test (A5.11), the linear-cost-scaling test (A5.12), the conflict-coexistence test (A5.13), the composition-requirements test (A5.14), the pattern-mapping test (A5.15), and the reproducibility test (A5.16). Each test verifies one architectural commitment specifically; comprehensive verification runs the whole sequence. None of the subsequent tests subsumes this one, and this one does not subsume any of them; that is the point of running them as standalone procedures rather than as a single composite check.

Subsequent work that adopts, extends, or argues against the CKS inspect-right commitment should run the test in the form formalized here, or name the difference if a different procedure is used. A deployment that claims inspect-right satisfaction without producing a passing run of this test is asserting a property the architecture has no procedural evidence for.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Verifying the Inspect Right: A Standalone Operational Test Procedure for the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.